Introdução

Apresentação da Disciplina

0004617A/4710A – Estrutura de Dados I

Prof. Me. Giovani Candido

Prof. Me. Luiz Fernando Sementille

BCC/BSI - Bauru

Ementa

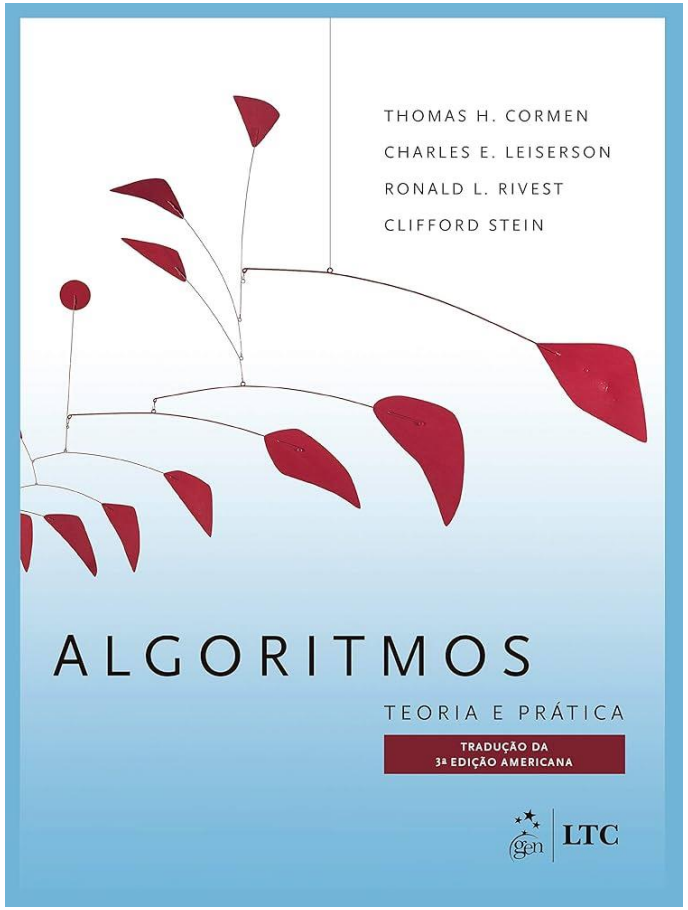
Assuntos

1. Introdução (TAD, ED e Listas)
2. Pilhas
3. Filas
4. Árvores (Generalizadas, Binárias, Binárias de Busca e AVL)
5. Heap
6. Hashing
7. Coleta e Compactação

Bibliografia

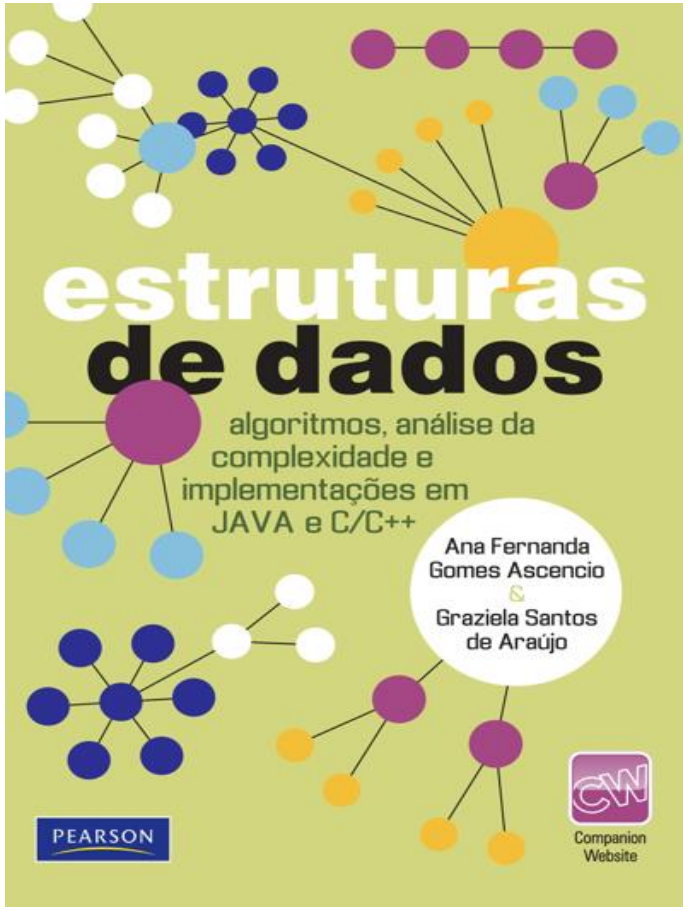
Bibliografia Básica

CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Editora Campus, v. 2, p. 296, 2002.



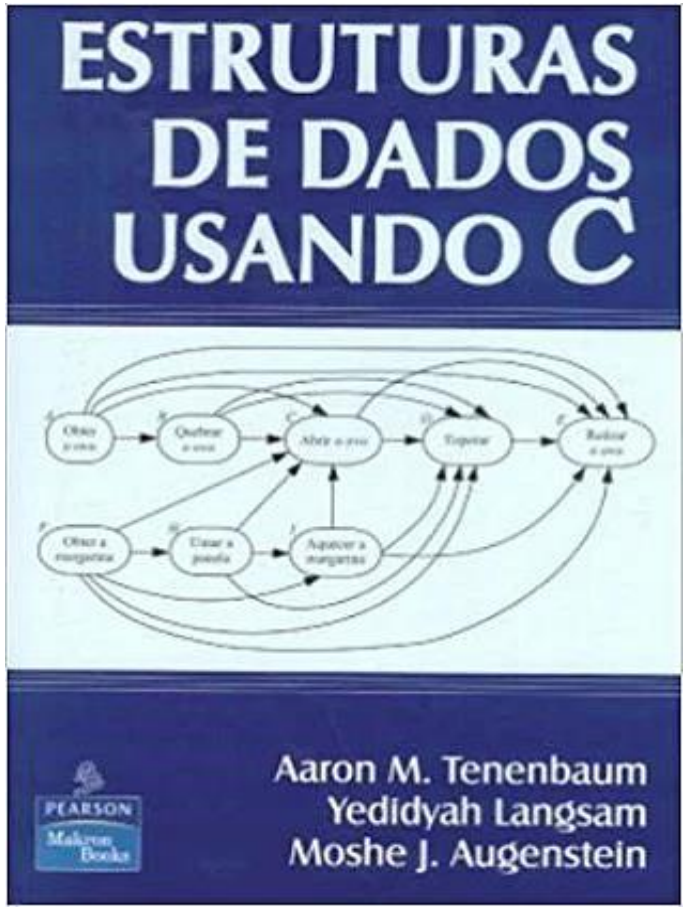
Bibliografia Básica

ASCENCIO, Ana Fernanda; ARAUJO, Graziela Santos. Estrutura de dados–Algoritmos. Análise de Complexidade e implementações em JAVA e C/C+.



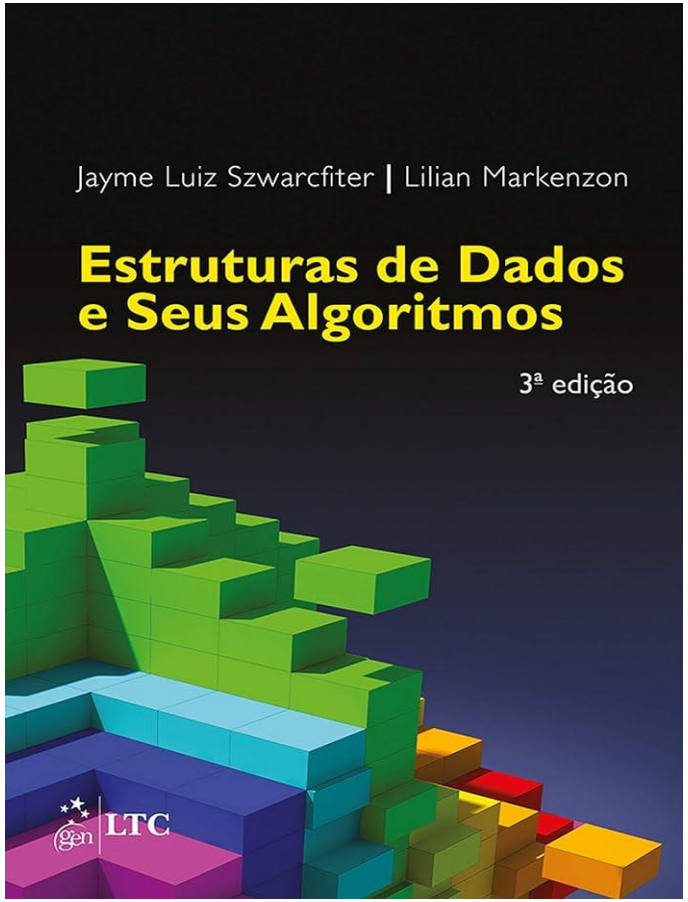
Bibliografia Básica

TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. Estruturas de dados usando C. Pearson Makron Books, 2004.



Bibliografia Complementar

SZWARCFILTER, Jaime Luis. Markenzon, Lilian. Estruturas de Dados e Seus Algoritmos, v. 2, 1994.



Bibliografia Complementar

DROZDEK, Adam. Estrutura de Dados e Algoritmos em C++. Cengage, 2002.

Bibliografia Complementar

CASTILHO, Marcos; SILVA, Fabiano; WEINGAERTNER, Daniel. Algoritmos e estruturas de dados I. Universidade Federal do Paraná. Curitiba, p. 345, 2020.

Metodologia

Metodologia

- Aulas expositivas e práticas:
  - Conteúdo teórico com implementação em linguagem *C.*
  - Resolução de exercícios sobre o conteúdo.
- Material disponibilizado durante ou após a aula.

Recursos Complementares

Visualização

- Data Structure Visualizations.
- VisuAlgo.

Visualização

Data Structure Visualizations

About
Algorithms
F.A.Q
Known Bugs / Feature Requests
Java Version
Flash Version
Create Your Own / Source Code
Contact

David Galles
Computer Science
University of San Francisco

Currently, we have visualizations for the following data structures and algorithms:

- Basics
  - Stack: Array Implementation
  - Stack: Linked List Implementation
  - Queues: Array Implementation
  - Queues: Linked List Implementation
  - Lists: Array Implementation (available in java version)
  - Lists: Linked List Implementation (available in java version)
- Recursion
  - Factorial
  - Reversing a String
  - N-Queens Problem
- Indexing
  - Binary and Linear Search (of sorted list)
  - Binary Search Trees
  - AVL Trees (Balanced binary search trees)
  - Red-Black Trees
  - Splay Trees
  - Open Hash Tables (Closed Addressing)
  - Closed Hash Tables (Open Addressing)
  - Closed Hash Tables, using buckets
  - Trie (Prefix Tree, 26-ary Tree)
  - Radix Tree (Compact Trie)
  - Ternary Search Tree (Trie with BST of children)
  - B Trees
  - B+ Trees

Visualização

Sites Online Judge

- Beecrowd.
- LeetCode.

Sites Online Judge

stack | Problems | SEARCH

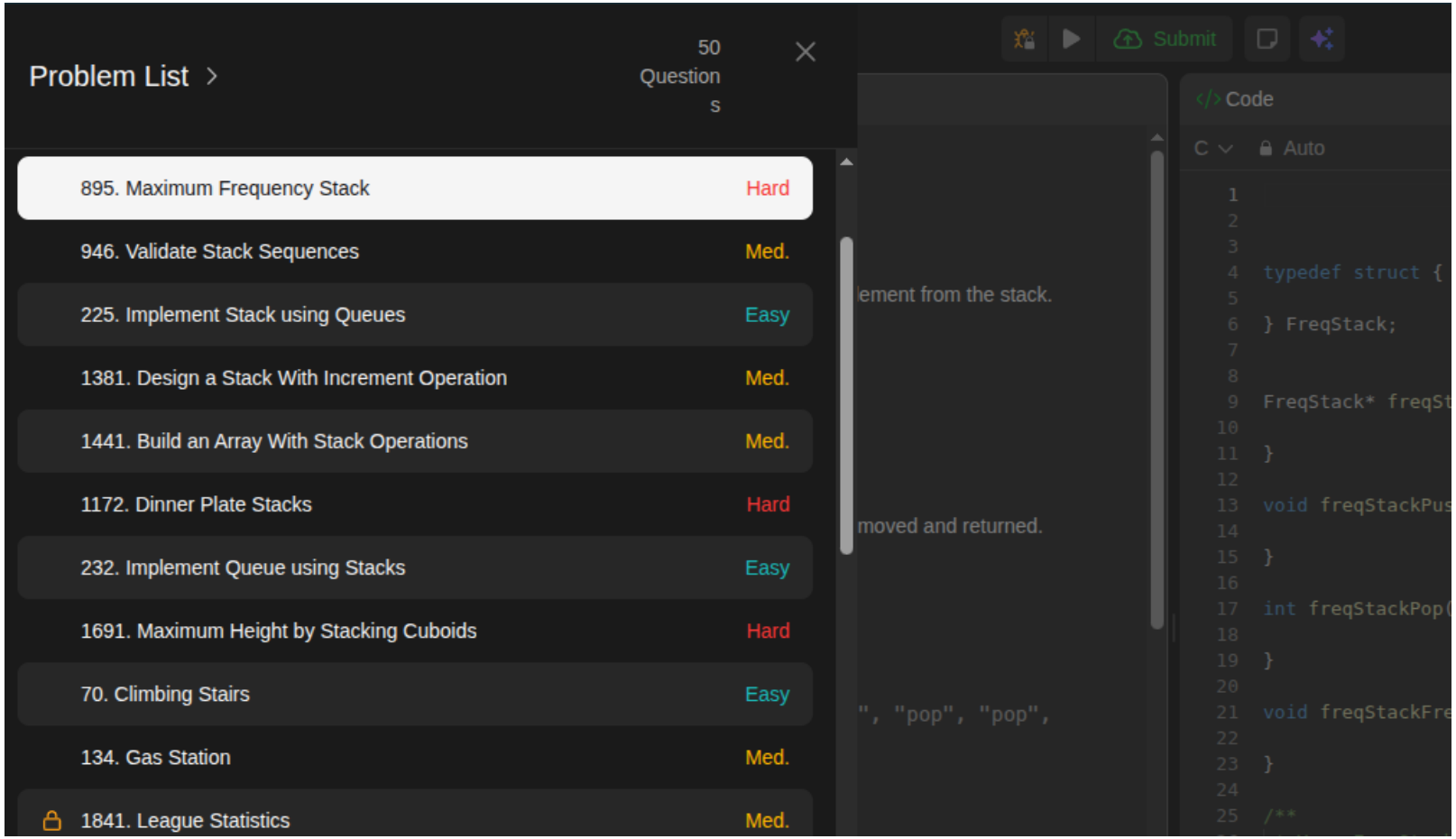
Back

Search Results

Results from problems

#	NAME	SUBJECT	FAVORITE	SOLVED	LEVEL
1062	Rails	**Stack**		9,231	5
1063	Rails Again... Tracing Movements	**Stack**		4,387	4
1068	Parenthesis Balance I	**Stack**		25,566	5
1069	Diamonds and Sand	**Stack**		24,036	3
1077	Infix to Posfix	**Stack**		5,606	5
1083	LEXSIM - Sintatic and Lexical Avaliator	**Stack**		1,275	7
1242	Alien Ribonucleic Acid	**Stack, String**		2,064	4

Sites Online Judge



Outros Recursos

- **Sites de Programação:**
  - Implementações alternativas em sites como GeeksforGeeks.
  - Soluções para problemas específicos em fóruns como StackOverflow.
- **IAs Generativas:**
  - Com cautela, para discutir conteúdo e validar conhecimento.
  - **Evitar**: implementações e resolução de exercícios ou erros de código.
- Materiais de outras universidades, videoaulas abertas etc.

Critérios de Avaliação

Critérios de Avaliação

- Serão avaliados com trabalhos e provas.
- Inicialmente, teremos:
  - Entrega de trabalhos com exercícios dos tópicos.
  - Duas provas escritas, dividindo o conteúdo da disciplina.

Critérios de Avaliação

Dessa forma, temos que:

$$NS = 0{,}7 \times MP + 0{,}3 \times MT$$

$$MP = \frac{P1 + P2}{2} \qquad MT = \frac{T1 + \cdots + Tt}{Nt}$$

Onde:

- ***NS*** é a Nota do Semestre.
- ***MP*** e ***MT*** são, respectivamente, as médias das provas, trabalhos e exercícios.

Critérios de Avaliação

- Terá direito a realizar o **exame**, o aluno que:
  - Obtiver **NS < 5.0.**
  - Tiver **Frequência >= 70%.**
- Com a aplicação do exame, temos:

$$NF = \frac{NS + NE}{2}$$

Onde:

- ***NF*** é a Nota Final;
- ***NS*** e ***NE*** são, respectivamente, as notas do semestre e do exame.

- Se a **NF >= 5.0**, o aluno será **aprovado**.

Ficou alguma dúvida?

E-mail para contato: giovani.candido@unesp.br e lf.sementille@unesp.br

Materiais e Avisos: Google Classroom